Project Settings Recommendations

FemboyEngine2D 0.5.0 - by game scale: light, medium, large

Every project descriptor (project.femproj) mixes three kinds of settings: gameplay simulation, shipping quality and build policy. The same numbers do not fit a two-week jam and a streaming open world. This guide gives one recommendation column per scale, explains the cost behind each value, and lists the mistakes that show up in profiles and bug reports. Treat the values as starting points and measure before shipping.

Three scale profiles

Profile	Typical game	Assets	Target
Light	Game jam, visual novel, puzzle, small pixel platformer	< 200 MB	60 FPS at 1080p on integrated graphics, CPU frame < 6 ms
Medium	Commercial metroidvania, roguelike, action platformer	0.2-1 GB	60 FPS at 1440p on a mid-range GPU, CPU frame < 8 ms
Large	Open world, large streaming tilemaps, heavy zoom and pan	1-8 GB	60+ FPS at 4K on a discrete GPU, CPU frame < 12 ms, VT

At a glance

Setting	Light	Medium	Large
fixed_update_hz	60	120	120 (240 for fighting/rhythm)
max_fixed_steps	4	8	8
master/sfx/music	1.0 / 1.0 / 1.0	1.0 / 1.0 / 1.0	1.0 / 1.0 / 1.0, mix at asset level
blind_mode / deaf_mode	false, options menu	false, options menu	false, options menu
flash_guard	true	true	true
flash_threshold / flash_gain	0.20 / 0.35	0.20 / 0.35	0.15 / 0.25 for strict safety
oled_pixel_shift / interval	true / 60 s	true / 60 s	true / 30-60 s
texture_format (art-driven)	rgba8 pixel art; bc7 painted	bc7-srgb; rgba8 for pixel art	bc7-srgb desktop; astc4x4-srgb on ASTC-capable GPUs
max_texture_size	1024	2048	4096
build.optimization	O2	O2	O3 (measure O2+LTO first)
build.lto	false for iteration	true for release	true for release
build.strip_debug_info	true for shipping	true for shipping	true for shipping
build.unity_build	true	true	true; disable only for build memory
build.native_arch	false	false	false
collision layers	3-4 used	up to 6 used	named set, unused pairs off
physics_backend	Auto	Auto	Auto or DOD for 4096+ simple bodies

Setting	Light	Medium	Large
physics_body_budget	0	0	50000 for strategy, 0 for general games
virtual_texture.enabled	false	true for zoom-heavy maps	true
VT page/tile/layers/table	-	1024/64/2/128	2048/64/4/256
VT budget / prefetch	-	6 / 1	8 / 2
editor.ui_scale	1.0	1.0-1.25	1.0-1.25
render.vsync	on	on	on; off only for latency profiling
editor.window_mode	1 borderless	1 borderless	1 borderless; 2 for perf checks

Simulation

fixed_update_hz

The fixed step is the physics and gameplay tick. 60 Hz is enough for puzzles, visual novels and slow platformers; 120 Hz pays off for precise action, ropes, projectiles and anything where 16 ms of input latency is felt. Fighting and rhythm games can justify 240 Hz, but then every fixed system must stay inside ~4 ms. The render rate is independent: never tie simulation to the monitor refresh.

max_fixed_steps

The catch-up cap for a slow frame. 4 is fine for light scenes; 8 is the usual commercial value. Raising it beyond 8 turns a single hitch into a spiral of death: the simulation runs more steps, the frame gets slower and the next frame needs even more steps. Lower it, do not raise it, if you see that pattern in a profile.

physics_backend / physics_body_budget

Auto keeps Box2D for joints, CCD and general rigid bodies and switches to the data-oriented backend only when the world is large and joint-free. Set the backend explicitly only after profiling. The body budget is a policy hint, not a hard limit: 4096 starts the Auto switch, 50000 is a strategy-world target. DOD bodies are simple AABB shapes without joints; use swept projectiles instead of CCD.

Audio

audio master / sfx / music

Keep the defaults at 1.0 and build headroom into the assets: mix music around -6 dBFS and leave peaks for explosions and UI. Boosting the master bus above 1.0 in the project to compensate for quiet assets clips the output and defeats the accessibility and deaf-mode paths. Use the buses for balance and a limiter on the master.

Accessibility

flash_guard, flash_threshold, flash_gain

Keep the flash guard on for every shipping profile. A luminance delta threshold of 0.20 and a gain of 0.35 stop most rapid flashes without flattening normal effects; strict safety settings are 0.15 and 0.25. Expose the toggle in the options menu but default it to on, and do not disable it in release builds for a 'punchier' look.

oled_pixel_shift / oled_interval_seconds

Leave the shift on for static HUDs, menus and long sessions. 60 seconds is a safe default; 30-60 seconds suits OLED monitors and handhelds. The cost is a hidden pixel offset, not a layout change, so it never affects gameplay logic.

blind_mode / deaf_mode

Default both to off and expose them as options. Blind mode rewires the UI to a screen-reader-friendly flow and deaf mode mutes the master bus at the graph root, so they must be tested with real content: narration text, subtitle timing and visual cues for audio events.

Textures and rendering

texture_format

Compression choice follows the art, not the production scale: a jam game with a large painted background benefits from BC7, and a large pixel-art game should still use RGBA8. Scale only changes max_texture_size and whether VT is needed. BC7 is the best general-purpose desktop codec in the engine's closed format set; it is not a 'medium' feature.

Format	Bits per pixel	Best for	Avoid	Availability
RGBA8	32	Pixel art, crisp UI, masks, small atlases	Large painted 4K art (memory)	Everywhere
BC7 sRGB/UNORM	8	Painted 2D art, gradients, alpha cutouts	1-pixel pixel art (edge bleed)	Desktop (D3D11+/Vulkan)
ASTC 4x4	8	Painted art on ASTC-capable GPUs, same quality class as BC7	GPUs without ASTC	Apple and many modern GPUs
ASTC 6x6	3.56	Size-first assets on ASTC-capable GPUs	Fine detail, text, UI	Apple and many modern GPUs

The format set is closed: RGBA8, BC7 and ASTC 4x4/6x6 only. BC3, BC4, BC5, BC6H and Basis/KTX2 are not cooked today; add them only with a concrete asset class that needs them (BC4/BC5 for single/two-channel masks, BC6H for HDR, Basis for download size with transcoding).

What looks better is often not the codec: mip generation, atlas padding (gutters) and sRGB handling remove bleeding and edge artifacts that no codec fixes. The practical next step for a mixed project is a per-asset format override, so an atlas can stay RGBA8 while a painted background uses BC7; today the project setting is the default for all cooked textures.

max_texture_size

Match the asset pipeline, not the monitor resolution. 1024 is plenty for a 1080p pixel game; 2048 covers most commercial 2D art at 1440p; 4096 is for 4K painted backgrounds and world atlases. Every doubling quadruples memory: a 4K RGBA8 page is 64 MB, with mips about 85 MB, while BC7 is about 16 MB and 21 MB with mips.

Collision

collision matrix

Start from the named layers and disable every pair the game does not use. Each enabled pair is broadphase and narrowphase work; in a bullets-and-pickups game the matrix can outweigh the level geometry. Light games need 3-4 layers, commercial titles up to 6, and large productions should keep the reserved layers but switch unused pairs off. Re-check the matrix after every new gameplay system.

Build and release

optimization

O2 is the sane default for iteration and most shipping builds; O3 adds code size and compile time with diminishing returns. For large projects compare O2+LTO against O3 with a real profile: LTO usually wins more than the optimisation level. Never ship an O0 build.

build.lto

Off while iterating: LTO makes incremental links slow. On for release builds. If link memory is a problem on CI, use thin LTO or keep unity builds on and split the largest translation units.

strip_debug_info

Strip for distribution, keep the symbols archive next to the release: crash reports without symbols cannot be triaged. The engine keeps the unstripped archive available for the same build; do not strip it.

unity_build

On by default: the engine's ~120 translation units build faster and link smaller. Turn it off only when a single huge unit exceeds memory or when a file needs isolated compile diagnostics; then expect slower iteration.

native_arch

Never enable it for a distributed build: binaries compiled for the host CPU can trap with an illegal instruction on older machines. It is a local benchmarking switch only.

Virtual texturing

VT is opt-in and only pays off when the world is larger than VRAM. Light games should leave it off. Medium games enable it for zoom-heavy tilemaps; large games depend on it. The layout is cooked into the chunks, so change it before the content is baked, not after.

Setting	Light	Medium	Large
virtual_texture.enabled	false	true for zoom-heavy maps	true
layers	-	2	4
page_size / tile_size	-	1024 / 64	2048 / 64
chunk_tiles	-	2	4
page_table_size	-	128	256
world_tile_size	-	256	256
budget_loads_per_frame	-	6	8
prefetch_tiles	-	1	2

Editor settings

Editor settings are fixed in a separate document: md/en/EditorSettingsReference_EN.md (Editor Settings Reference) lists every key with its type, default, clamp, the editor.ini mirror and the single settings file behind them. The short version: keep UI scale at 1.0-1.25, keep vsync on for stable authoring frame pacing (turn it off only for latency profiling), keep asset watching and auto-refresh on in a team so the browser reflects a colleague's import, and use borderless fullscreen (window_mode 1) for daily work; exclusive fullscreen (2) is for performance measurements.

Recommended presets

Light

```
[simulation]
fixed_update_hz = 60.0
max_fixed_steps = 4

[render]
texture_format = "rgba8"
max_texture_size = 1024

[build]
optimization = "O2"
lto = false
strip_debug_info = true
unity_build = true
native_arch = false

[virtual_texture]
enabled = false
```

Medium

```
[simulation]
fixed_update_hz = 120.0
max_fixed_steps = 8
```

```
[render]
texture_format = "bc7-srgb"
max_texture_size = 2048

[build]
optimization = "O2"
lto = true
strip_debug_info = true
unity_build = true
native_arch = false

[virtual_texture]
enabled = true
layers = 2
page_size = 1024
tile_size = 64
chunk_tiles = 2
page_table_size = 128
world_tile_size = 256.0
budget_loads_per_frame = 6
prefetch_tiles = 1
```

Large

```
[simulation]
fixed_update_hz = 120.0
max_fixed_steps = 8

[render]
texture_format = "bc7-srgb"
max_texture_size = 4096

[build]
optimization = "O3"
lto = true
strip_debug_info = true
unity_build = true
native_arch = false

[virtual_texture]
enabled = true
layers = 4
page_size = 2048
tile_size = 64
chunk_tiles = 4
page_table_size = 256
world_tile_size = 256.0
budget_loads_per_frame = 8
prefetch_tiles = 2
```

Common mistakes

- Light: 4K textures, VT, O3+LTO, 240 Hz physics - cost without visible benefit.
- Light and medium: BC7 for pixel art - soft edges and palette shifts.
- Choosing the codec by project scale instead of art style - a painted jam game still wants BC7.

- Any scale: native_arch in a distribution build - illegal instructions on older CPUs.
- Large: one VT layer with a 2048 page - constant eviction and blurred pages.
- Large: a huge page table instead of shifting its origin - wasted memory.
- Shipping with streaming disabled or the flash guard off.

Verification checklist

- The profile matches the target frame time at the shipping resolution.
- Fixed-step count stays below max_fixed_steps in the worst scene.
- Texture memory fits the budget with mips (measure, do not estimate).
- Collision pairs are all reachable by gameplay and none are unused.
- Release uses LTO, stripped binaries and keeps the symbols archive.
- Accessibility toggles work with content, not only with placeholders.
- VT (if enabled) shows no deferred backlog and no magenta tiles in steady state.

Related documentation

See the project issue tracker for the forward plan, the Application Guide for the editor-to-release workflow and the Complete Reference for the settings architecture.